GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Construcción de software				
TÍTULO DE LA PRÁCTICA:	**UML como código**				
NÚMERO DE PRÁCTICA:	05	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	VI
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		3
FECHA INICIO:	05/10/2026	**FECHA FIN:**	10/10/2026	**DURACIÓN:**	90 min presenciales + trabajo autónomo

RECURSOS A UTILIZAR:

Computador con acceso a Internet; cuenta de GitHub y el repositorio del Laboratorio 04; Visual Studio Code con las extensiones PlantUML y Markdown Preview Mermaid Support; PlantUML (Java 17+ o servidor web plantuml.com); Mermaid Live Editor (mermaid.live); Python 3.10+ con pylint (incluye pyreverse) o IntelliJ IDEA para Java; asistente de IA generativa (Claude, ChatGPT, Gemini o GitHub Copilot); aula virtual del curso.

DOCENTE(s):

Mg. Antonio Arroyo Paz

OBJETIVOS/TEMAS Y COMPETENCIAS

OBJETIVOS:

1. Modelar la estructura de un módulo de software con diagramas de clases y de paquetes coherentes con la arquitectura definida en el Laboratorio 04.
2. Modelar el comportamiento de un flujo crítico con diagramas de secuencia, de máquina de estados y de actividades.
3. Escribir los diagramas UML como código (PlantUML y Mermaid), versionados en Git.
4. Aplicar ingeniería directa e inversa (round-trip) con apoyo de IA y verificar la consistencia entre diagramas y código.

TEMAS:

- UML 2.5.1: diagramas estructurales y de comportamiento; UML como boceto y como plano.
- Diagrama de clases: atributos, operaciones, visibilidad, multiplicidad y relaciones.
- Diagramas de secuencia (fragmentos alt, opt y loop), de máquina de estados y de actividades (particiones, fork/join).
- Diagrama de paquetes y reglas de dependencia.
- UML como código, ingeniería directa e inversa, y reglas de consistencia entre diagramas, verificadas con apoyo de IA.

COMPETENCIAS	**C.b:** Genera de forma responsable prototipos, experimentos y modelos, con el fin de analizar e interpretar información para la toma de decisiones fundamentadas y objetivas.

	C.c: Diseña responsablemente sistemas, componentes o procesos para satisfacer necesidades dentro de restricciones realistas.
	C.d: Trabaja éticamente, de manera efectiva, en equipos multidisciplinarios, adaptándose a diferentes entornos y nuevas situaciones.
	C.g: Comunica con claridad y de forma efectiva ideas, resultados y productos relacionados con su ejercicio profesional.

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

1.1 Del diseño arquitectónico al diseño detallado

En el Laboratorio 04 se decidió **cómo se organiza el sistema** (por ejemplo, un monolito modular con sus módulos). En este laboratorio se diseña **el interior de los módulos**: qué clases existen, cómo colaboran para resolver una historia de usuario, qué estados atraviesa una entidad y qué reglas de dependencia respetan los paquetes. El lenguaje estándar para hacerlo es **UML 2.5.1** (OMG, 2017), que define 14 tipos de diagramas.

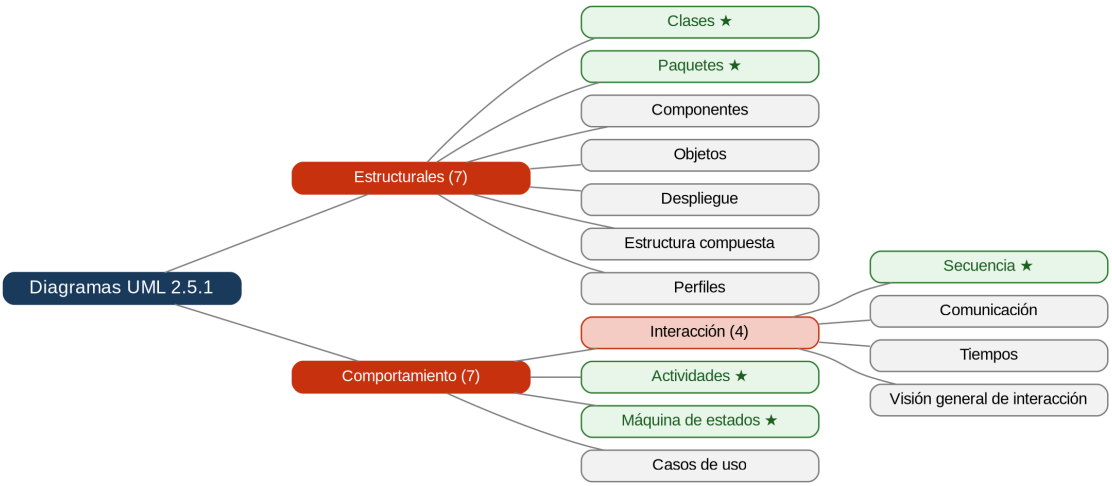


Figura 1. *Taxonomía de los diagramas UML 2.5.1. Con ★ se marcan los cinco diagramas que se trabajan en esta guía. Diagrama generado con Graphviz (DOT).*

Fowler (2003) distingue tres usos de UML: como **boceto** (para pensar y comunicar), como **plano** (especificación detallada) y como **lenguaje de programación**. En equipos ágiles predomina el uso como boceto y como plano de las partes críticas: se modela **lo suficiente** para tomar buenas decisiones y no se documenta todo el sistema.

1.2 Diagrama de clases

Muestra las clases, sus **atributos** y **operaciones** con su visibilidad (`+` pública, `-` privada, `#` protegida, `~` de paquete) y las **relaciones** entre ellas, con su **multiplicidad** (`1`, `0..1`, `0..*`, `1..*`).

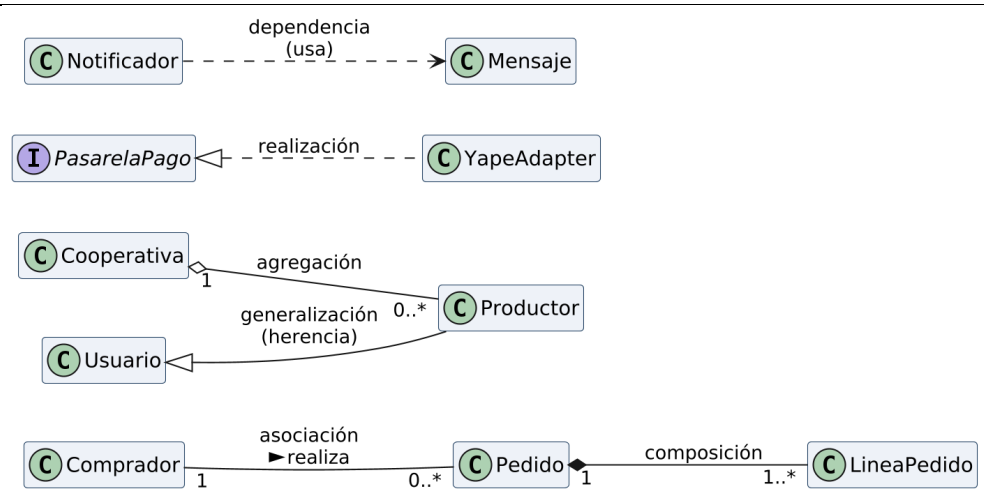


Figura 2. *Notación de las relaciones del diagrama de clases (renderizado con PlantUML).*

Tabla 1. Relaciones del diagrama de clases

Relación	Significado	Ejemplo en AgroConecta	PlantUML
Asociación	Una clase conoce a otra de forma estable.	Un comprador realiza pedidos.	`A "1" -- "0..*" B`
Agregación	Todo–parte débil: la parte puede existir sin el todo.	Una cooperativa agrupa productores.	`A o-- B`
Composición	Todo–parte fuerte: la parte nace y muere con el todo.	Un pedido contiene sus líneas.	`A *-- B`
Generalización	Herencia: "es un".	Un productor es un usuario.	`A <|-- B`
Realización	Una clase implementa una interfaz.	YapeAdapter implementa PasarelaPago.	`I <|.. C`
Dependencia	Uso temporal (parámetro, variable local).	El notificador usa un mensaje.	`A ..> B`

1.3 Diagrama de secuencia

Describe **cómo colaboran los objetos en el tiempo** para resolver un escenario: líneas de vida, mensajes síncronos (flecha llena), asíncronos (flecha abierta) y de retorno (punteada), y **fragmentos combinados**: `alt` (alternativas), `opt` (opcional), `loop` (repetición) y `par` (paralelo). Cada mensaje dirigido a un objeto debe corresponder a una **operación de su clase**.

1.4 Diagrama de máquina de estados

Modela el **ciclo de vida** de una entidad: estados, transiciones con la forma `evento [guarda] / acción`, estado inicial y estados finales. Es muy útil para entidades como un pedido, una cita o un préstamo, y de él se derivan directamente **casos de prueba** (una prueba por transición).

1.5 Diagrama de actividades

Representa un **proceso o flujo de trabajo**: acciones, decisiones, bifurcaciones paralelas (fork/join) y **particiones** (swimlanes) que indican quién realiza cada acción. Es útil para validar procesos de negocio con usuarios no técnicos.

1.6 Diagrama de paquetes

Agrupa las clases en paquetes (que corresponden a los módulos del Laboratorio 04) y muestra sus **dependencias**. Su regla más importante es que **no existan dependencias cíclicas**: si dos paquetes dependen mutuamente, en la práctica forman un solo módulo.

1.7 UML como código, round-trip e IA

Igual que en el Laboratorio 04, los diagramas se escriben como texto. **PlantUML** soporta todos los diagramas UML de esta guía; **Mermaid** soporta clases, secuencia y estados, y GitHub lo renderiza directamente. Además:

- **Ingeniería directa:** a partir del diagrama de clases se genera el esqueleto del código. La IA lo hace en segundos, pero puede cambiar nombres, omitir relaciones o agregar lógica no pedida.
- **Ingeniería inversa:** a partir del código se genera el diagrama. Herramientas como **pyreverse** (incluida en pylint, para Python) o IntelliJ IDEA (para Java) lo hacen automáticamente.
- **Round-trip:** comparar el diagrama original con el que se obtiene del código revela inconsistencias que de otro modo pasarían desapercibidas.

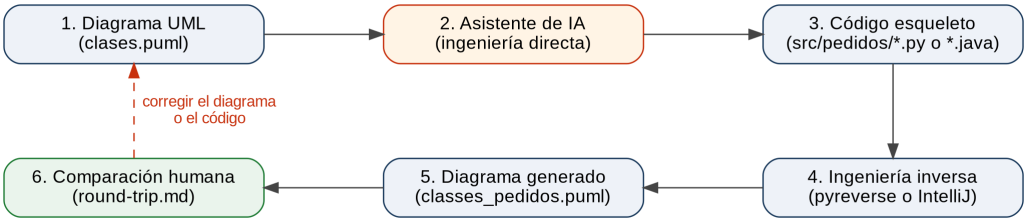


Figura 3. *Flujo round-trip asistido por IA que se aplica en esta guía. Diagrama generado con Graphviz.*

Tabla 2. Reglas de consistencia entre diagramas (se usan en los ejercicios)

Regla	Consistencia que se verifica
C1	Cada mensaje del diagrama de secuencia dirigido a un objeto es una **operación** de la clase de ese objeto en el diagrama de clases.
C2	Cada transición del diagrama de estados está provocada por una **operación** o un evento identificable de la clase.
C3	Las multiplicidades reflejan las reglas del negocio (por ejemplo, un pedido tiene al menos una línea: `1..*`).
C4	Las dependencias entre paquetes respetan el ADR-001 del Laboratorio 04 y **no forman ciclos**.
C5	Los nombres usan el **lenguaje del dominio** del caso y son iguales en diagramas, código e historias de usuario.

Regla de oro de esta práctica

La IA propone, el equipo decide y verifica.
Todo diagrama o código generado por IA se revisa con las reglas C1–C5,
se corrige si es necesario y se registra en la bitácora de IA.
Nunca ingrese datos personales o confidenciales en un prompt.

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Caso: AgroConecta Majes (continuación del Laboratorio 04). En el ADR-001 se eligió un **monolito modular** con los módulos Catálogo, Pedidos, Pagos, Notificaciones y Reportes, con integraciones mediante **puertos y adaptadores**. El MVP debe salir en **1 mes**. Ahora se diseña el interior de los módulos **Pedidos y Pagos**.

Paso 1. Elegir el alcance: una historia de usuario crítica

docs/design/historia.md

```
HU-07: Como comprador, quiero confirmar mi pedido y pagarlo con Yape o Plin
       para no manejar efectivo.

Criterios de aceptación
- Dado un pedido en BORRADOR con stock suficiente, cuando lo confirmo y el pago
  es aprobado, entonces el pedido queda PAGADO y el productor recibe un WhatsApp.
- Dado un pedido confirmado, cuando el pago es rechazado o no hay respuesta en
  15 min, entonces el pedido queda CANCELADO y el stock reservado se libera.
```

Paso 2. Generar un primer diagrama de clases con IA

PROMPT IA 1 — Diagrama de clases

Actúa como diseñador de software orientado a objetos. Contexto: módulo Pedidos y Pagos de un monolito modular (ADR-001: puertos y adaptadores para integraciones externas). Historia y criterios: [pegar historia.md].
Tarea: genera un diagrama de clases en PlantUML con atributos tipados, operaciones, multiplicidades, una enumeración para el estado del pedido y una interfaz (puerto) para la pasarela de pagos.
Formato: solo el código PlantUML. No agregues clases que no se deriven de la historia; si asumes algo, indícalo en un comentario.

Verificación del docente sobre la primera respuesta de la IA:

Tabla 3. Bitácora de IA: primer diagrama de clases

Propuesta de la IA	Decisión	Justificación
Atributo `numeroTarjeta` en Pago	Rechazada	La historia solo usa Yape/Plin; además, guardar tarjetas exige cumplir PCI DSS.
`Yape extends Pago` (herencia)	Corregida	Contradice el ADR-001: se modela PasarelaPago como interfaz (puerto) con adaptadores que la realizan.
Relación Pedido–LineaPedido como asociación simple	Corregida	Las líneas no existen sin su pedido: es una composición `1` a `1..*` (regla C3).
Enumeración EstadoPedido	Aceptada	Coincide con los criterios de aceptación.

Paso 3. Diagrama de clases final (PlantUML)

PlantUML — docs/design/clases.puml

```
@startuml clases-pedidos
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam defaultFontName Arial
skinparam classBackgroundColor #EEF2F8
skinparam classBorderColor #1A3A5C
```

```
title AgroConecta — Módulos Pedidos y Pagos (diseño)
skinparam nodesep 25
skinparam ranksep 40

class Comprador {
  - id: UUID
  - razonSocial: String
  - distrito: String
}
class Productor {
  - id: UUID
  - nombre: String
  - celular: String
}
class Oferta {
  - id: UUID
  - producto: String
  - precioUnitario: Decimal
  - stockDisponible: int
  + reservar(cantidad: int): void
  + liberar(cantidad: int): void
}
class Pedido {
  - id: UUID
  - fecha: DateTime
  - estado: EstadoPedido
  + agregarLinea(oferta: Oferta, cantidad: int): void
  + calcularTotal(): Decimal
  + confirmar(): void
  + registrarPago(pago: Pago): void
  + cancelar(motivo: String): void
}
class LineaPedido {
  - cantidad: int
  - precioUnitario: Decimal
  + subtotal(): Decimal
}
enum EstadoPedido {
  BORRADOR
  CONFIRMADO
  PAGO_PENDIENTE
  PAGADO
  EN_PREPARACION
  ENTREGADO
  CANCELADO
}
class Pago {
  - id: UUID
  - monto: Decimal
  - medio: MedioPago
  - codigoOperacion: String
  - aprobado: bool
}
enum MedioPago {
  YAPE
  PLIN
}
interface PasarelaPago <<puerto>> {
  + cobrar(monto: Decimal, celular: String): Pago
}
class YapeAdapter <<adaptador>>
class PlinAdapter <<adaptador>>
class ServicioPedidos <<servicio>> {
  + confirmarYPagar(pedidoId: UUID, medio: MedioPago): Pedido
}
```

```
interface RepositorioPedidos <<puerto>> {
  + buscar(id: UUID): Pedido
  + guardar(p: Pedido): void
}
interface Notificador <<puerto>> {
  + notificarNuevoPedido(p: Pedido): void
}

Comprador "1" -- "0..*" Pedido : realiza >
Pedido "1" *-- "1..*" LineaPedido
LineaPedido "0..*" --> "1" Oferta
Productor "1" -- "0..*" Oferta : publica >
Pedido "1" -- "0..1" Pago
Pedido --> EstadoPedido
Pago --> MedioPago
PasarelaPago <|.. YapeAdapter
PasarelaPago <|.. PlinAdapter
ServicioPedidos ..> Pedido
ServicioPedidos ..> RepositorioPedidos
ServicioPedidos ..> PasarelaPago
ServicioPedidos ..> Notificador
@enduml
```

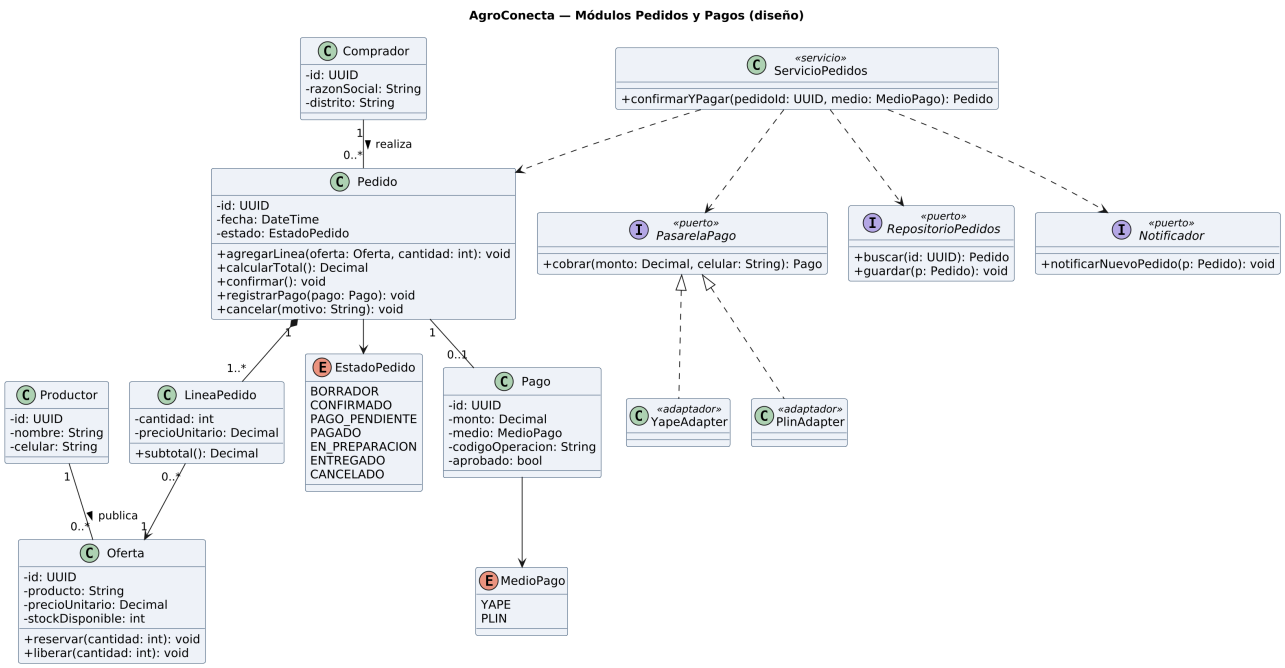


Figura 4. *Diagrama de clases de los módulos Pedidos y Pagos (renderizado con PlantUML).*

Paso 4. Diagrama de secuencia del flujo "confirmar y pagar"

PlantUML — docs/design/secuencia-confirmar-pagar.puml

```
@startuml secuencia-confirmar-pagar
skinparam shadowing false
skinparam defaultFontName Arial
skinparam sequenceMessageAlign center
autonumber
actor Comprador
participant "PWA" as UI
participant "PedidoController" as C
participant "ServicioPedidos" as S
participant "RepositorioPedidos" as R
```

```
participant "Pedido" as P
participant "PasarelaPago\n(YapeAdapter)" as G
participant "Notificador" as N

Comprador -> UI : Confirmar y pagar con Yape
UI -> C : POST /pedidos/{id}/pago
C -> S : confirmarYPagar(id, YAPE)
S -> R : buscar(id)
R --> S : pedido
S -> P : confirmar()
loop por cada línea
  P -> P : oferta.reservar(cantidad)
end
S -> P : calcularTotal()
P --> S : total
S -> G : cobrar(total, celular)
alt pago aprobado
  G --> S : pago (aprobado)
  S -> P : registrarPago(pago)
  S -> R : guardar(pedido)
  S ->> N : notificarNuevoPedido(pedido)
  note right of N : asíncrono\n(no bloquea la respuesta)
  S --> C : pedido PAGADO
  C --> UI : 200 OK
else pago rechazado o sin respuesta
  G --> S : pago (rechazado)
  S -> P : cancelar("pago rechazado")
  S -> R : guardar(pedido)
  S --> C : pedido CANCELADO
  C --> UI : 402 Pago rechazado
end
UI --> Comprador : Muestra el resultado
@enduml
```

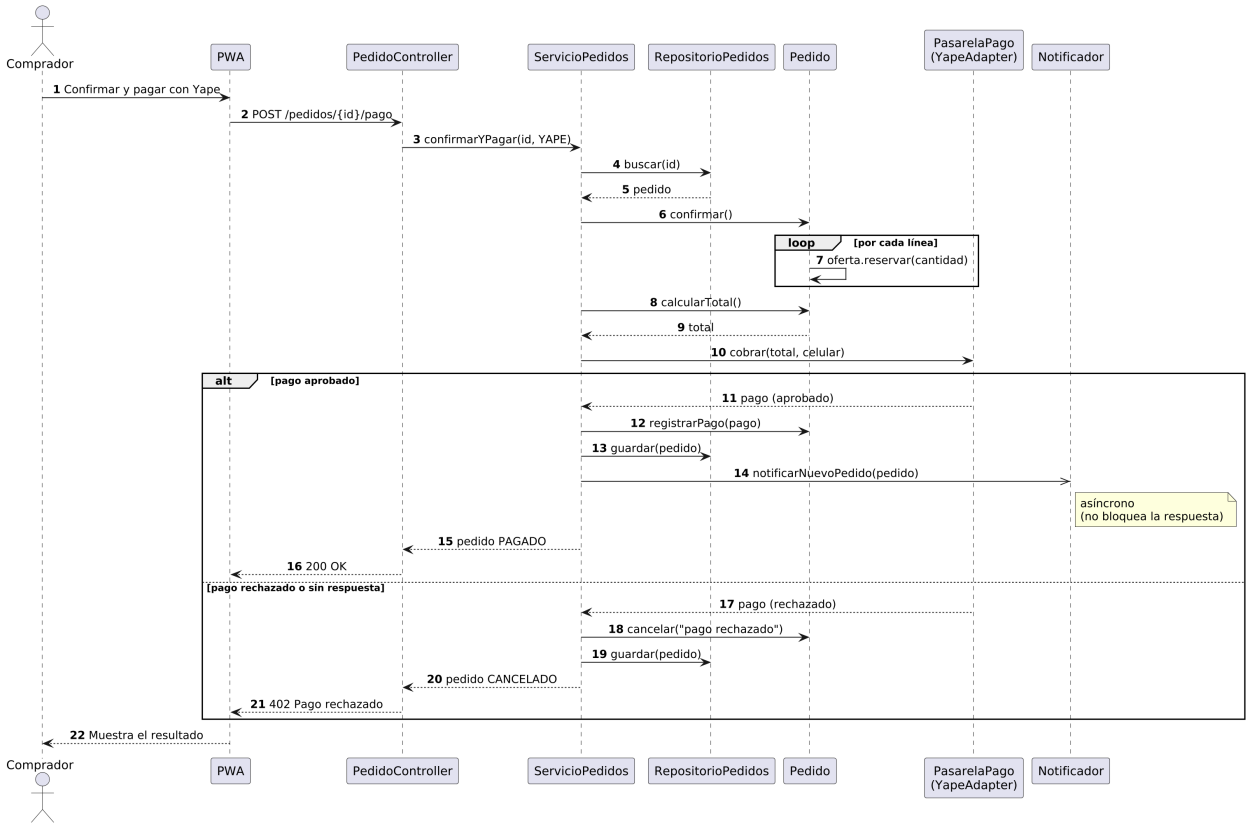


Figura 5. *Diagrama de secuencia con fragmentos loop y alt, y un mensaje asíncrono (renderizado con PlantUML).*

Verificación C1: cada mensaje enviado a `Pedido` (confirmar, calcularTotal, registrarPago, cancelar), a `PasarelaPago` (cobrar), a `RepositorioPedidos` (buscar, guardar) y a `Notificador` (notificarNuevoPedido) existe como operación en el diagrama de clases.

Paso 5. Diagrama de máquina de estados del Pedido (Mermaid)

Mermaid — docs/design/estados-pedido.mmd

```
stateDiagram-v2
    [*] --> BORRADOR : agregarLinea()
    BORRADOR --> CONFIRMADO : confirmar() [stock suficiente]
    CONFIRMADO --> PAGO_PENDIENTE : cobrar()
    PAGO_PENDIENTE --> PAGADO : registrarPago() [aprobado]
    PAGO_PENDIENTE --> CANCELADO : cancelar() [rechazado o 15 min sin respuesta]
    PAGADO --> EN_PREPARACION : productor acepta
    EN_PREPARACION --> ENTREGADO : confirmarEntrega()
    BORRADOR --> CANCELADO : cancelar()
    CONFIRMADO --> CANCELADO : cancelar()
    ENTREGADO --> [*]
    CANCELADO --> [*]
    note right of PAGADO
        Al entrar: notificar al productor
    end note
```

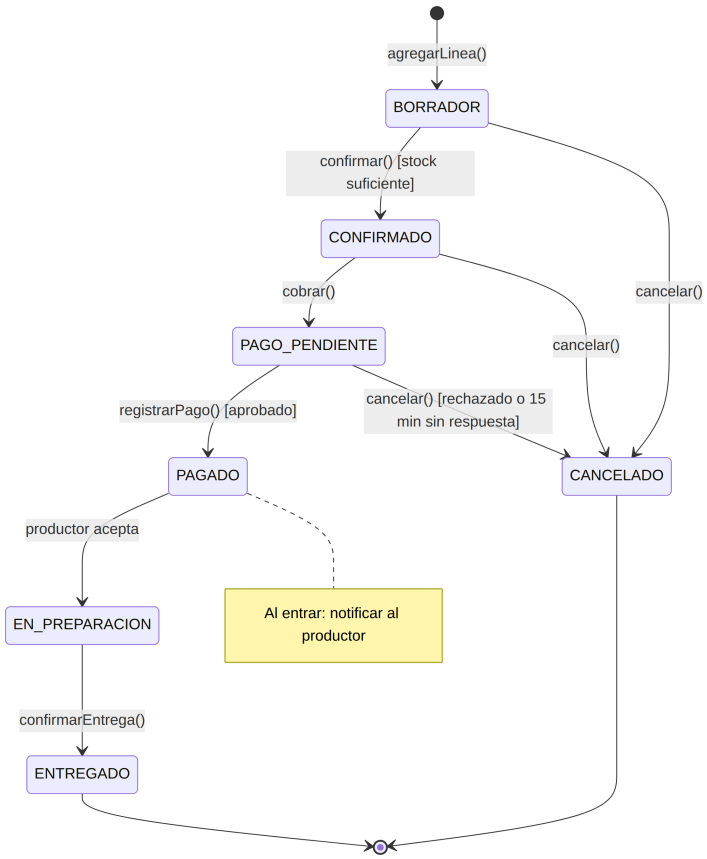


Figura 6. *Máquina de estados del Pedido (renderizada con mermaid-cli).*

Verificación C2: las transiciones usan operaciones de la clase Pedido. La transición "productor acepta" no tenía operación, por lo que se agregó `aceptarPreparacion()` a la lista de cambios pendientes del diagrama de clases.

Paso 6. Diagrama de actividades "atender un pedido" (PlantUML)

PlantUML — docs/design/actividades-atender-pedido.puml

```
@startuml actividades-atender-pedido
skinparam shadowing false
skinparam defaultFontName Arial
skinparam ActivityBackgroundColor #EEF2F8
skinparam ActivityBorderColor #1A3A5C
|Comprador|
start
:Arma el carrito y confirma;
|Sistema|
:Valida el stock de cada oferta;
if (¿Stock suficiente?) then (sí)
  :Reserva el stock;
  :Cobra con Yape / Plin;
  if (¿Pago aprobado?) then (sí)
    fork
      :Notifica al productor\npor WhatsApp;
    fork again
      :Envía el comprobante\nal comprador;
    end fork
  else (no)
    :Libera el stock;
    |Comprador|
    :Recibe aviso de pago rechazado;
```

```
    stop
  endif
else (no)
  |Comprador|
  :Ajusta cantidades;
  stop
endif
|Productor|
:Prepara el pedido;
:Entrega en el punto acordado;
|Comprador|
:Confirma la recepción;
stop
@enduml
```

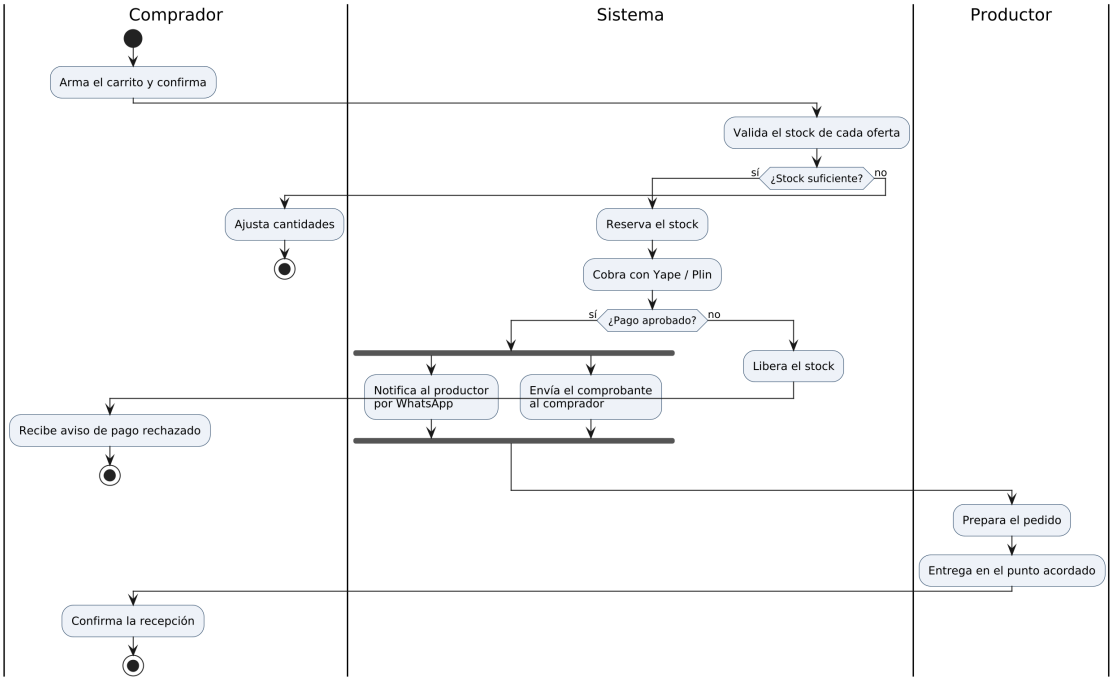


Figura 7. *Diagrama de actividades con tres particiones, decisiones y fork/join.*

Paso 7. Diagrama de paquetes (coherencia con el ADR-001)

PlantUML — docs/design/paquetes.puml

```
@startuml paquetes
skinparam shadowing false
skinparam defaultFontName Arial
skinparam packageStyle rectangle
title AgroConecta — Paquetes del monolito modular (vista de desarrollo)
package "agroconecta" {
  package "catalogo" as CAT
  package "pedidos" as PED
  package "pagos" as PAG
  package "notificaciones" as NOT
  package "reportes" as REP
  package "compartido" as COM
}
PED ..> CAT : usa Oferta
PED ..> PAG : usa PasarelaPago\n(puerto)
```

```
PED ..> NOT : usa Notificador\n(puerto)
REP ..> PED : solo lectura
CAT ..> COM
PED ..> COM
PAG ..> COM
NOT ..> COM
note bottom of PAG
  Regla (ADR-001): sin dependencias cíclicas.
  pagos NO depende de pedidos.
end note
@enduml
```

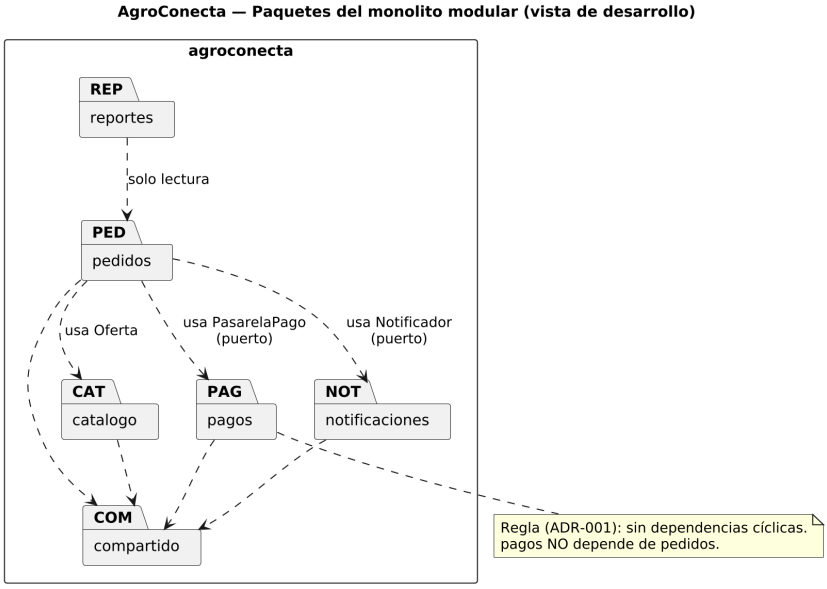


Figura 8. *Diagrama de paquetes: dependencias sin ciclos (regla C4).*

Paso 8. Round-trip: del diagrama al código y de vuelta

PROMPT IA 2 — Ingeniería directa

Genera el esqueleto en Python 3.10 (dataclasses y type hints) del siguiente diagrama de clases. Respeta exactamente los nombres de clases, atributos y operaciones (en snake_case), las enumeraciones, la interfaz como clase abstracta (ABC) y las multiplicidades. Implementa solo la lógica mínima de las operaciones del Pedido; los adaptadores deben lanzar NotImplementedError. [pegar clases.puml]

Python — src/pedidos/dominio.py (extracto del código generado y revisado)

```
class EstadoPedido(Enum):
    BORRADOR = "BORRADOR"
    CONFIRMADO = "CONFIRMADO"
    ...

@dataclass
class Pedido:
    lineas: list[LineaPedido] = field(default_factory=list)
    estado: EstadoPedido = EstadoPedido.BORRADOR
    pago: Pago | None = None
    id: UUID = field(default_factory=uuid4)
```

```
    def confirmar(self) -> None:
        for linea in self.lineas:
            linea.oferta.reservar(linea.cantidad)
        self.estado = EstadoPedido.CONFIRMADO
    ...

class PasarelaPago(ABC):
    @abstractmethod
    def cobrar(self, monto: Decimal, celular: str) -> Pago: ...

class YapeAdapter(PasarelaPago):
    def cobrar(self, monto: Decimal, celular: str) -> Pago:
        raise NotImplementedError("Integrar con el proveedor de pagos")
```

Bash — ingeniería inversa con pyreverse

```
pip install pylint
pyreverse -o puml -p pedidos src/pedidos      # genera classes_pedidos.puml y packages_pedidos.puml
java -jar plantuml.jar -tpng classes_pedidos.puml
```

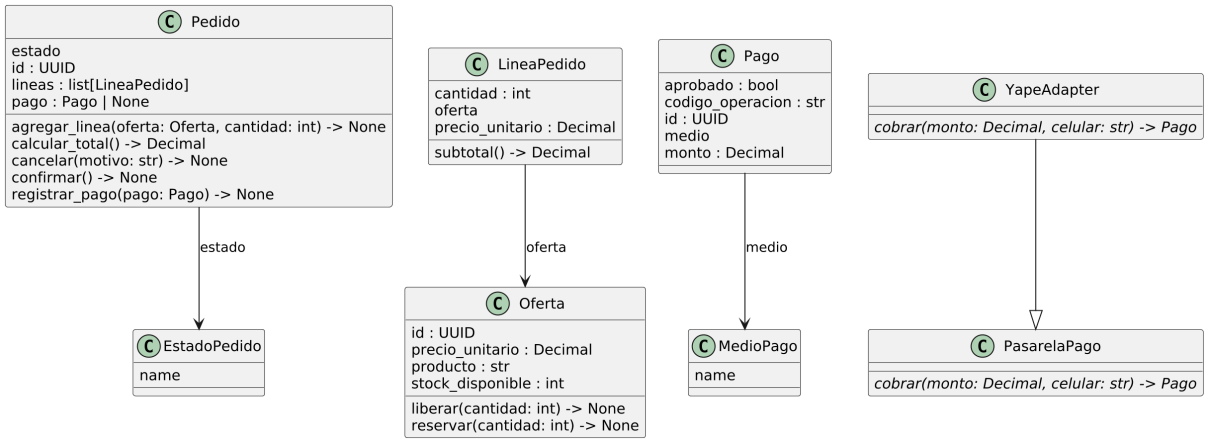


Figura 9. *Diagrama de clases obtenido automáticamente del código con pyreverse (ingeniería inversa).*

Tabla 4. Extracto de round-trip.md: comparación entre el diagrama de diseño y el diagrama obtenido del código

Diferencia observada	Causa	Acción
No aparece la composición Pedido–LineaPedido; las líneas figuran como atributo `list[LineaPedido]`.	pyreverse no infiere asociaciones desde colecciones tipadas.	Ninguna: limitación de la herramienta; el diagrama de diseño se mantiene.
No hay multiplicidades.	El código no expresa multiplicidades.	Agregar una validación en `confirmar()`: el pedido debe tener al menos una línea (C3).
PasarelaPago aparece como clase, no como interfaz, y `cobrar()` de YapeAdapter figura como abstracto.	Python implementa interfaces con ABC; el adaptador aún lanza NotImplementedError.	Aceptable en Python; se documenta.
Faltan Comprador, Productor, ServicioPedidos y los puertos RepositorioPedidos y Notificador.	La IA generó solo una parte del diagrama.	Corregir: pedir a la IA las clases faltantes y regenerar (bitácora).

Nombres en snake_case (`calcular_total`) frente a camelCase en el diagrama.	Convención de Python.	Aceptable; se documenta la equivalencia (C5).

Paso 9. Revisión de consistencia con IA

PROMPT IA 3 — Auditor de consistencia

Actúa como revisor de diseño. Te paso cinco diagramas UML en PlantUML/Mermaid: [pegar los cinco archivos]. Verifica estas reglas y responde en una tabla (regla, elemento, problema, corrección sugerida): C1 cada mensaje de secuencia es una operación de la clase receptora; C2 cada transición de estados corresponde a una operación de la clase; C3 multiplicidades coherentes con los criterios de aceptación; C4 paquetes sin ciclos; C5 nombres consistentes. No reescribas los diagramas; solo reporta hallazgos y cita la línea.

Tabla 5. Extracto de consistencia.md

Regla	Hallazgo de la IA	Verificación del equipo
C2	La transición "productor acepta" no corresponde a ninguna operación.	Correcto: se agrega `aceptarPreparacion()` a Pedido.
C1	`oferta.reservar()` se invoca desde Pedido, pero no existe la asociación Pedido–Oferta.	Falso positivo: Pedido accede a Oferta a través de LineaPedido (navegación válida).
C4	"pagos depende de pedidos".	Falso: la IA leyó al revés la flecha `PED ..> PAG`. Se rechaza el hallazgo.

Paso 10. Estructura del repositorio y contenido de cada archivo

Se trabaja en el **mismo repositorio del Laboratorio 04**, en una carpeta nueva `docs/design/`. Los códigos **E1 a E8** corresponden a los **entregables** de los ejercicios propuestos (E1 = Ejercicio 1, etc.).

Estructura del repositorio cs-2026b-lab04-grupoXX (continuación)

```
cs-2026b-lab04-grupoXX/
├── README.md                          # E8: agregar la sección "Diseño UML (Lab 05)"
├── docs/
│   ├── architecture/                  # Laboratorio 04 (solo se corrige si hace falta)
│   └── design/                        # Laboratorio 05
│       ├── historia.md                # E1: historia de usuario y criterios
│       ├── clases.puml                # E1: diagrama de clases
│       ├── secuencia-<flujo>.puml     # E2: diagrama de secuencia
│       ├── estados-<entidad>.mmd      # E3: máquina de estados (Mermaid)
│       ├── actividades-<proceso>.puml # E4: diagrama de actividades
│       ├── paquetes.puml              # E5: diagrama de paquetes
│       ├── round-trip.md              # E6: tabla de diferencias código-diagrama
│       ├── consistencia.md            # E7: hallazgos C1–C5
│       ├── bitacora-ia.md             # E7: interacciones con la IA
│       └── img/                       # PNG/SVG de todos los diagramas
└── src/<modulo>/                      # E6: esqueleto de código generado
```

Tabla 6. Qué va dentro de cada archivo

Código	Archivo	Contenido mínimo obligatorio	Modelo
E1	historia.md, clases.puml	Historia con 2 o más criterios de aceptación. Diagrama con 6 o más clases, 1 interfaz, 1 enumeración, atributos tipados con visibilidad, operaciones y multiplicidades en todas las asociaciones.	Pasos 1–3
E2	secuencia-<flujo>.puml	5 o más líneas de vida, un fragmento `alt` y uno `loop` u `opt`, al menos un mensaje asíncrono y los mensajes de retorno.	Paso 4
E3	estados-<entidad>.mmd	5 o más estados, guardas entre corchetes, estado inicial y final, y transiciones nombradas con operaciones de la clase.	Paso 5
E4	actividades-<proceso>.puml	3 o más particiones (swimlanes), 2 o más decisiones y un fork/join.	Paso 6
E5	paquetes.puml	Paquetes iguales a los módulos del ADR-001, dependencias etiquetadas y una nota con la regla de dependencias.	Paso 7
E6	src/<modulo>/, round-trip.md	Código generado con IA, diagrama obtenido por ingeniería inversa y tabla con 3 o más diferencias (causa y acción).	Paso 8
E7	consistencia.md, bitacora-ia.md	Tabla de hallazgos C1–C5 (al menos 1 falso positivo identificado) y bitácora con 5 o más interacciones.	Pasos 2 y 9
E8	README.md + tag	Sección "Diseño UML (Lab 05)" con los diagramas embebidos y enlaces; etiqueta `v0.5-diseno` en GitHub.	Sección III

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Organización: se mantienen los **mismos grupos y el mismo caso del Laboratorio 04**, y se trabaja en el mismo repositorio (`cs-2026b-lab04-grupoXX`). Así, el diseño detallado debe ser coherente con su arquitectura y su ADR-001. Si el grupo cambió de caso, debe completar primero `drivers.md` del Laboratorio 04.

Condiciones comunes

- Plazo: el MVP debe estar en producción en 1 mes; modele solo lo necesario para la historia crítica.
- Herramientas: PlantUML para clases, secuencia, actividades y paquetes; Mermaid para estados.
- Regla de oro: la IA propone, el equipo decide y verifica; todo se registra en bitacora-ia.md.

Foco de diseño sugerido para cada caso

Tabla 7. Foco de diseño por caso (los casos son los mismos del Laboratorio 04)

N.º	Caso	Historia / flujo para la secuencia (E2)	Entidad para estados (E3)	Proceso para actividades (E4)
1	CitaSalud Arequipa	Reservar una cita verificando cupo y evitando doble asignación	Cita: Reservada, Confirmada, Atendida, NoAsistió, Cancelada	Atención del día: admisión, médico y paciente
2	PicanteríaYa	Pedir el menú del día y pagar con Yape	Pedido: Recibido, Preparando, EnCamino, Entregado, Cancelado	Asignación del repartidor más cercano
3	AireMisti IoT	Ingesta de una lectura y emisión de alerta	Sensor: Activo, SinSeñal, Sincronizando, Mantenimiento, Baja	Emisión y difusión de una alerta pública
4	BiblioUNSA	Reservar un libro validando la matrícula vigente	Préstamo: Reservado, Prestado, Vencido, Devuelto, ConMulta	Devolución con cálculo de multa

5	VotoEPIS	Emitir un voto secreto	Elección: Configurada, Abierta, Cerrada, Escrutada, Publicada	Escrutinio y firma del acta
6	RutaSIT Arequipa	Consultar el tiempo de llegada del bus	Viaje: Programado, EnRuta, Desviado, Detenido, Finalizado	Detección y aviso de un desvío
7	San Camilo en Línea	Pedido a varios puestos con pago único	PedidoPuesto: Recibido, Listo, Recogido, Entregado, Rechazado	Preparación y consolidación de un pedido multipuesto
8	SismoReporta AQP	Enviar un reporte sin señal y sincronizarlo	Reporte: Borrador, Encolado, Recibido, Verificado, Atendido	Priorización y asignación de brigadas
9	ChacraSmart Majes	Programar un riego y abrir la válvula de forma remota	Válvula: Cerrada, Abriendo, Abierta, Cerrando, Falla (cierre seguro)	Riego automático según la humedad del suelo
10	EcoRecicla AQP	Solicitar el recojo de reciclables	Solicitud: Pendiente, Asignada, Recogida, Pesada, PuntosAcreditados	Ruta diaria del reciclador

E1. Historia de usuario y diagrama de clases

1. En `historia.md`, escriba la historia de usuario de la columna E2 de su caso, con **2 o más criterios de aceptación** en formato Dado / Cuando / Entonces.
2. Adapte el **Prompt IA 1** con su historia y su ADR-001, y genere un primer diagrama de clases. Registre el prompt en `bitacora-ia.md`.
3. Revise la propuesta: elimine clases que no se deriven de la historia, corrija relaciones (composición o asociación, herencia o realización) y agregue las multiplicidades.
4. El diagrama final debe tener: **6 o más clases**, **1 interfaz** (puerto hacia un servicio externo), **1 enumeración**, atributos con tipo y visibilidad, y operaciones.
5. Renderice con PlantUML, guarde el PNG en `img/` y haga commit con el mensaje `E1: diagrama de clases`.

☐ **Lista de verificación:** todas las asociaciones tienen multiplicidad en ambos extremos; ningún atributo almacena datos sensibles innecesarios; los nombres son del dominio del caso (C5).

E2. Diagrama de secuencia del flujo crítico

1. Elija el escenario principal de su historia (por ejemplo, "reservar cita con cupo disponible").
2. Dibuje en PlantUML **5 o más líneas de vida**: actor, interfaz (UI o API), controlador, servicio, entidades y puertos.
3. Incluya un fragmento **`alt`** para el caso de éxito y el de error, y un fragmento **`loop`** u **`opt`**.
4. Incluya al menos un **mensaje asíncrono** (`->>`), por ejemplo una notificación, y los mensajes de retorno.
5. Verifique la regla **C1**: si un mensaje no existe como operación, agréguela al diagrama de clases. Haga commit con el mensaje `E2: secuencia`.

☐ **Lista de verificación:** el diagrama cubre el criterio de aceptación de error; cada mensaje se corresponde con una operación (C1).

E3. Máquina de estados de la entidad principal

1. Tome la entidad sugerida para su caso y escriba el diagrama en Mermaid (`stateDiagram-v2`). Valídelo en mermaid.live.
2. Incluya **5 o más estados**, el estado inicial `[*]`, al menos un estado final y **2 o más guardas** entre corchetes.

3. Nombre cada transición con la operación de la clase que la provoca (C2). Haga commit con el mensaje `E3: estados`.

☐ **Lista de verificación:** no hay estados sin salida, salvo los finales; los estados coinciden con la enumeración del diagrama de clases.

Parte B — Trabajo autónomo (06/10/2026 al 10/10/2026)

E4. Diagrama de actividades

1. Modele el proceso de la última columna de la tabla de casos con PlantUML (`|Partición|`).
2. Incluya **3 o más particiones**, **2 o más decisiones** (`if / else`) y una ejecución paralela (`fork / end fork`).
3. Valide el flujo con alguien que conozca el proceso real (un compañero que actúe como usuario) y anote un cambio que surgió de esa revisión.

E5. Diagrama de paquetes

1. Cree un paquete por cada módulo del ADR-001, más un paquete `compartido` si lo necesita.
2. Dibuje y etiquete las dependencias (qué clase o puerto se usa) y agregue una nota con la regla de dependencias.
3. Verifique la regla **C4** (sin ciclos). Si encuentra un ciclo, resuélvalo con un puerto (interfaz) y registre la decisión en un nuevo ADR (`adr/004-...md`).

E6. Round-trip con IA

1. Adapte el **Prompt IA 2** y genere el esqueleto del código de su diagrama de clases en `src/<modulo>/`, en Python o Java.
2. **Python:** ejecute `pyreverse -o puml -p <modulo> src/<modulo>`. **Java:** use IntelliJ IDEA (Diagrams > Show Diagram) o el plugin PlantUML Parser y exporte el diagrama.
3. Compare ambos diagramas y registre en `round-trip.md` **3 o más diferencias**, con su causa y la acción tomada (corregir el diagrama, corregir el código o documentar la limitación).

E7. Revisión de consistencia y bitácora de IA

1. Aplique el **Prompt IA 3** a sus cinco diagramas y copie los hallazgos en `consistencia.md`.
2. Verifique cada hallazgo: **al menos uno** debe ser un falso positivo de la IA, explicado (como en el Paso 9).
3. Complete `bitacora-ia.md` con **5 o más interacciones** (fecha, herramienta, prompt, propuesta, verificación, decisión).

E8. README, revisión cruzada y entrega

1. Agregue al `README.md` la sección **"Diseño UML (Lab 05)"**, con el diagrama de estados embebido en Mermaid y enlaces a los demás diagramas e imágenes.
2. Cada integrante abre al menos **un Pull Request** con uno de los entregables, y otro integrante lo revisa aplicando las reglas C1–C5.
3. Cree la etiqueta de entrega: `git tag v0.5-diseno` y luego `git push origin v0.5-diseno`. Entregue el enlace del repositorio en el Aula Virtual hasta el **sábado 10/10/2026 a las 23:59**.

IV. CUESTIONARIO

1. ¿Qué diferencia hay entre el diseño arquitectónico del Laboratorio 04 y el diseño detallado de este laboratorio? Dé un ejemplo de su caso.
2. ¿Cuándo usaría composición y cuándo agregación? Justifique con dos relaciones de su diagrama de clases.

3. ¿Por qué en el diagrama de clases se modela la pasarela de pagos (o el servicio externo de su caso) como interfaz y no como superclase? Relaciónelo con el ADR-001.
4. Explique la regla C1 y muestre un mensaje de su diagrama de secuencia que la incumplía antes de corregirlo.
5. ¿Cómo se derivan casos de prueba a partir de un diagrama de máquina de estados? Indique cuántas pruebas mínimas requiere su diagrama.
6. ¿Qué información del diagrama de diseño se perdió en la ingeniería inversa (E6) y por qué?
7. ¿Qué falso positivo detectó en la revisión de consistencia con IA? ¿Qué habría pasado si lo aceptaba sin verificar?
8. Según Fowler, UML puede usarse como boceto, plano o lenguaje de programación. ¿Cuál de esos usos aplicó en este laboratorio y por qué es adecuado para un MVP de 1 mes?

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Fowler, M. (2003). *UML distilled: A brief guide to the standard object modeling language* (3rd ed.). Addison-Wesley.

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3rd ed.). Prentice Hall.

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language user guide* (2nd ed.). Addison-Wesley.

Valente, M. T. (2020). *Engenharia de Software Moderna*, cap. 4: Modelos. https://engsoftmoderna.info/cap4.html

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

PlantUML. (s. f.). *Documentación oficial*. https://plantuml.com/

Mermaid. (s. f.). *Documentación oficial: class, sequence y state diagrams*. https://mermaid.js.org/

Pylint. (s. f.). *Pyreverse: UML diagrams for Python code*. https://pylint.readthedocs.io/en/latest/additional_tools/pyreverse/index.html

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:	**INSTRUMENTOS:**
▪ Observación sistemática en el laboratorio (checkpoint E1–E3). ▪ Análisis de productos: diagramas como código, código y documentos del repositorio. ▪ Revisión entre pares mediante Pull Requests.	▪ Lista de cotejo del checkpoint. ▪ Rúbrica analítica (criterios abajo). ▪ Historial de commits, Pull Requests y etiqueta v0.5-diseno.

CRITERIOS DE EVALUACIÓN

Criterio	Descripción	Puntaje
E1 Diagrama de clases	Historia con criterios; 6 o más clases, interfaz, enumeración, relaciones correctas y multiplicidades.	4
E2 Diagrama de secuencia	5 o más líneas de vida, alt y loop/opt, mensaje asíncrono; cumple C1.	4
E3 Máquina de estados	5 o más estados, guardas, inicial y final; cumple C2.	2
E4–E5 Actividades y paquetes	Particiones, decisiones y fork/join; paquetes sin ciclos y coherentes con el ADR-001.	3
E6 Round-trip	Código generado, diagrama por ingeniería inversa y 3 o más diferencias analizadas.	3
E7 Consistencia y bitácora	Hallazgos C1–C5 con al menos un falso positivo; 5 o más interacciones con IA.	2
E8 Repositorio y entrega	README actualizado, Pull Requests revisados y etiqueta v0.5-diseño a tiempo.	2
Total		20